CNN Accelerator Subsystem with AXI4 Interface

Technical Report

Anonymous Submission

Internal Hackathon – Hardware / VLSI Division

Contents

1 System-Level Architecture **2**

2 CNN Accelerator Architecture **3**
2.1 Network Selection 4
2.2 Dataflow 4
2.3 Compute Engine 5
2.4 Memory Architecture 5
2.5 Fixed-Point Representation 5
2.6 Quantization Strategy 6
2.7 Performance vs Area Trade-offs 6

3 CNN Accelerator Microarchitecture **7**
3.1 MAC Unit 7
3.2 Requantizer 7
3.3 Pooling Engine 8
3.4 Control FSM 9

4 Memory Map **10**
4.1 Feature Map Memory 11
4.2 Future CDC-Compatible Memory Architecture 11

1 System-Level Architecture

The goal of this project was to design a complete CNN accelerator subsystem capable of performing inference on a small CNN model. Since this was my first accelerator design project and the development time was limited, I decided to focus on building a simple and fully functional architecture instead of targeting maximum performance.

The implemented network is a simplified Tiny-LeNet style architecture consisting of an 8×8 input image, a convolution layer, a max-pooling layer, and a fully connected classification layer producing four output classes. Figure 1 shows the overall organization of the proposed subsystem.

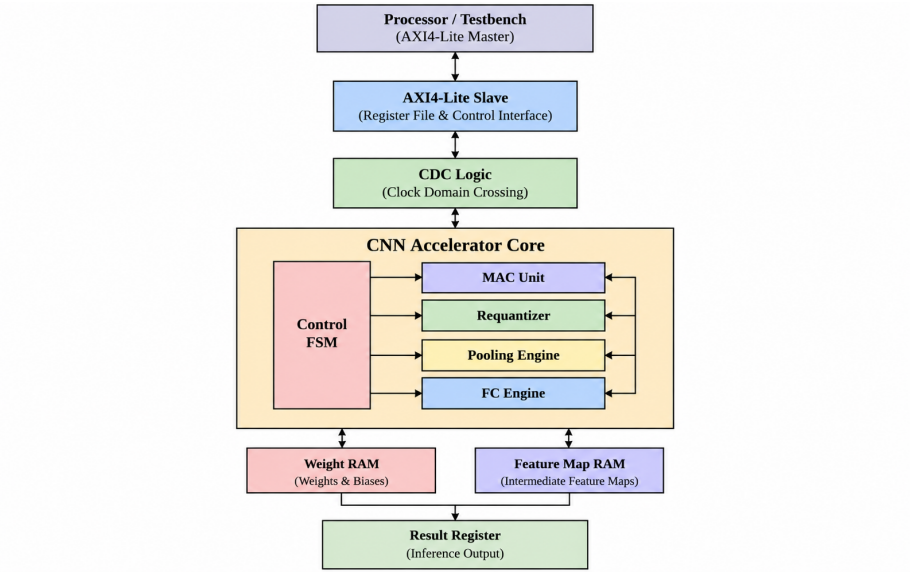


Figure 1: Top-level CNN accelerator subsystem architecture.

For communication with an external processor, an AXI4-Lite based interface was selected. Since the implemented network contains a relatively small amount of input data and parameters, using a full AXI4 or AXI4-Stream interface would increase design complexity without providing significant benefits. AXI4-Lite provides a simple mechanism for configuration, control, status monitoring, and result retrieval.

The CNN accelerator core is built around a single MAC unit that is reused for both convolution and fully connected computations. A parallel MAC architecture was considered, but for such a small network the additional hardware complexity and memory bandwidth requirements were not justified. Implementing multiple MAC units would also require a more complex memory subsystem capable of supporting multiple simultaneous reads, increasing resource utilization.

The memory subsystem stores convolution weights, fully connected weights, biases, intermediate feature maps, and pooling outputs. A centralized FSM controls memory accesses and schedules the execution of convolution, pooling, and fully connected operations.

To reduce unnecessary computations, a simple zero-gating mechanism is implemented. Whenever the activation value entering the MAC operation is zero, the multiplication and accumulation step is skipped. This avoids performing computations that do not contribute to the final output and reduces switching activity inside the datapath.

Overall, the architecture prioritizes simplicity, modularity, and ease of verification while still demonstrating the key concepts required in a CNN accelerator subsystem.

2 CNN Accelerator Architecture

This chapter describes the architecture of the implemented CNN accelerator. The design targets a simplified Tiny-LeNet style network consisting of an 8×8 input image, a convolution layer, a max-pooling layer, and a fully connected classification layer producing four output classes. The primary design objective was to build a simple and functional accelerator architecture that could be completed within the hackathon timeline while still demonstrating the key concepts of CNN hardware acceleration.

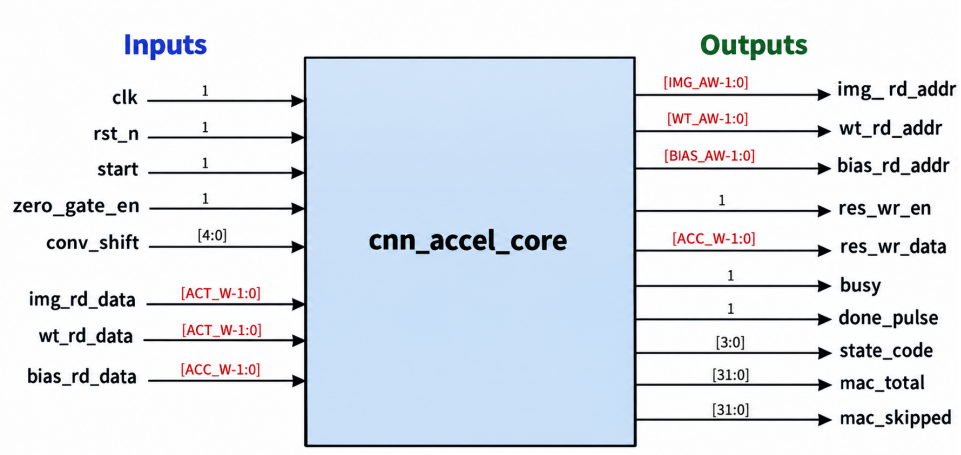


Figure 2: Top-level CNN accelerator core showing memory interfaces, control signals, and performance monitoring outputs.

The accelerator core interfaces with dedicated image, weight, and bias memories. A centralized FSM controls convolution, pooling, and fully connected execution while generating memory addresses and write operations.

The design also exposes status and performance signals such as `busy`, `done_pulse`, `mac_total`, and `mac_skipped` for debugging and evaluation.

2.1 Network Selection

Instead of implementing the complete LeNet-5 architecture, a simplified Tiny-LeNet style network was selected. This decision was made to reduce implementation complexity and allow more focus on accelerator architecture, memory organization, and control logic. The implemented network consists of:

- Input Image: 8×8
- Convolution Layer: 3×3 kernel
- Activation Function: ReLU
- Max-Pooling Layer: 2×2
- Fully Connected Layer
- Output Classes: 4

This architecture is sufficiently small to be executed using a single reusable MAC unit while still demonstrating the complete CNN inference flow.

2.2 Dataflow

The accelerator follows a sequential execution flow controlled by a centralized finite state machine (FSM).

The inference process is performed in the following order:

1. Load input image and network parameters.
2. Perform convolution.
3. Apply ReLU activation and requantization.
4. Store generated feature maps.
5. Execute max-pooling.
6. Flatten pooled outputs.
7. Perform fully connected computation.
8. Generate classification result.

Intermediate results are stored in on-chip memory and reused by subsequent stages.

2.3 Compute Engine

The compute engine is built around a single multiply-accumulate (MAC) unit.

A parallel MAC architecture was considered but was not selected because:

- The implemented network is relatively small.
- Multiple MAC units would require higher memory bandwidth.
- Additional memory ports would increase resource utilization.
- Design complexity would increase significantly.

The single MAC architecture provides a simple and resource-efficient solution while maintaining acceptable inference latency for the selected network size.

2.4 Memory Architecture

The design uses a shared memory structure for storing weights, biases, feature maps, and pooled outputs.

The memory layout is organized as follows:

Table 1: Memory Organization

Memory Region	Purpose
Weight Index + 0 to 8	Convolution Weights
Weight Index + 9 onward	Fully Connected Weights
Bias Index + 0 onward	Convolution Biases
Bias Index + 9 onward	Fully Connected Biases
Feature Map Region	Convolution Outputs
Pooling Region	Pooling Outputs

A centralized FSM generates all memory addresses required during inference and schedules read and write operations.

2.5 Fixed-Point Representation

Floating-point arithmetic was avoided to reduce hardware complexity and resource usage. All computations are performed using fixed-point integer arithmetic.

The fixed-point approach provides:

- Lower hardware cost

- Reduced memory usage
- Faster arithmetic operations
- Simpler RTL implementation

This representation is sufficient for the simplified Tiny-LeNet network implemented in this project.

2.6 Quantization Strategy

The accelerator uses quantized parameters and activations throughout the inference pipeline.

After accumulation, the output passes through:

1. Bias Addition
2. ReLU Activation
3. Arithmetic Right Shift
4. Saturation to 8-bit Range

This operation is implemented using a dedicated requantizer module, ensuring that intermediate values remain within the supported activation range.

2.7 Performance vs Area Trade-offs

Several architectural decisions were made with simplicity and area efficiency as the primary objectives.

Table 2: Major Design Trade-offs

Decision	Reason
Single MAC Unit	Reduced hardware complexity and area usage
AXI4-Lite Interface	Sufficient for small parameter and data sizes
Shared Memory Structure	Lower resource utilization
Fixed-Point Arithmetic	Simpler hardware implementation
Tiny-LeNet Network	Faster development and verification

The resulting architecture prioritizes simplicity, modularity, and ease of verification rather than maximum throughput. This makes it suitable as a reference implementation for understanding CNN accelerator design principles.

3 CNN Accelerator Microarchitecture

This chapter describes the internal hardware blocks that form the CNN accelerator core. The accelerator is built around a reusable single MAC datapath, a requantization unit, memory access logic, and a centralized control FSM. Each block is designed to minimize hardware complexity while supporting the complete inference flow of the implemented Tiny-LeNet network.

3.1 MAC Unit

The MAC (Multiply-Accumulate) unit is the primary compute block of the accelerator and is reused for both convolution and fully connected operations. A single MAC architecture was chosen to reduce hardware complexity and memory bandwidth requirements.

As shown in Figure 3, the MAC unit consists of input registers, a multiplier, a 32-bit accumulator, and control logic. Activation and weight values are multiplied and accumulated over multiple clock cycles to generate the final output. A simple zero-gating mechanism is also implemented to skip computations when the activation value is zero, reducing unnecessary operations.

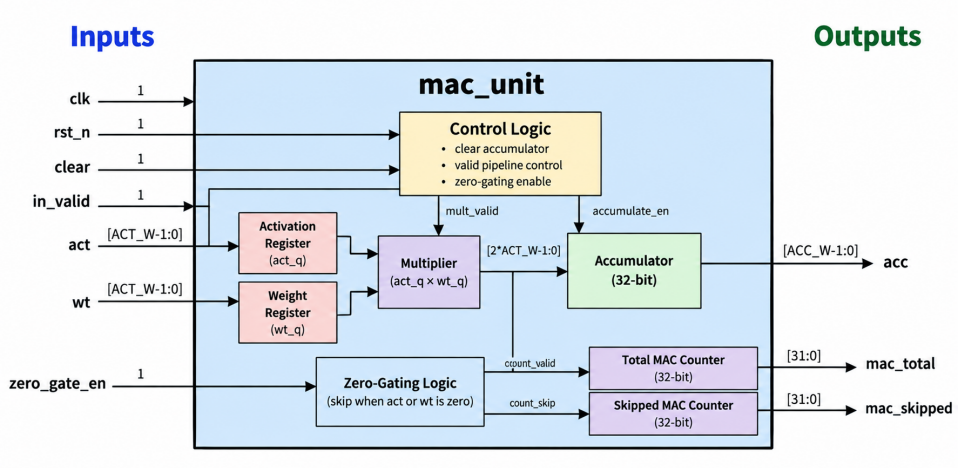


Figure 3: Internal architecture of the MAC unit.

3.2 Requantizer

The requantizer converts the 32-bit accumulated output of the MAC unit into an 8-bit activation value suitable for storage and further processing. As shown in Figure 4, the accumulated result first passes through bias addition, followed by an optional ReLU activation. The output is then scaled using an arithmetic right shift and finally saturated to the valid int8 range.

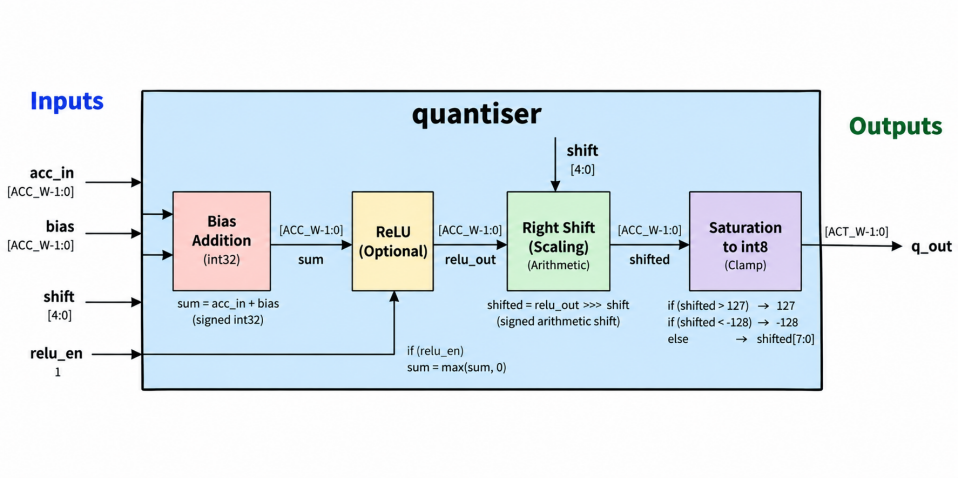


Figure 4: Internal architecture of the requantizer module.

3.3 Pooling Engine

The pooling engine performs a 2×2 max-pooling operation on the convolution output feature map. Since the feature map memory provides only a single read port, the pooling operation is performed sequentially by reading one value per cycle and maintaining a running maximum.

As shown in Figure 5, the pooling FSM controls the complete operation, including memory reads, address generation, comparison, and result storage. For each pooling window, four values are read from the feature map memory and compared using a running-max approach. After all four values have been processed, the maximum value is written back to the pooling region of memory.

This implementation avoids the need for multiple memory read ports or dedicated parallel comparison hardware, making it well suited for the resource-efficient architecture targeted in this project.

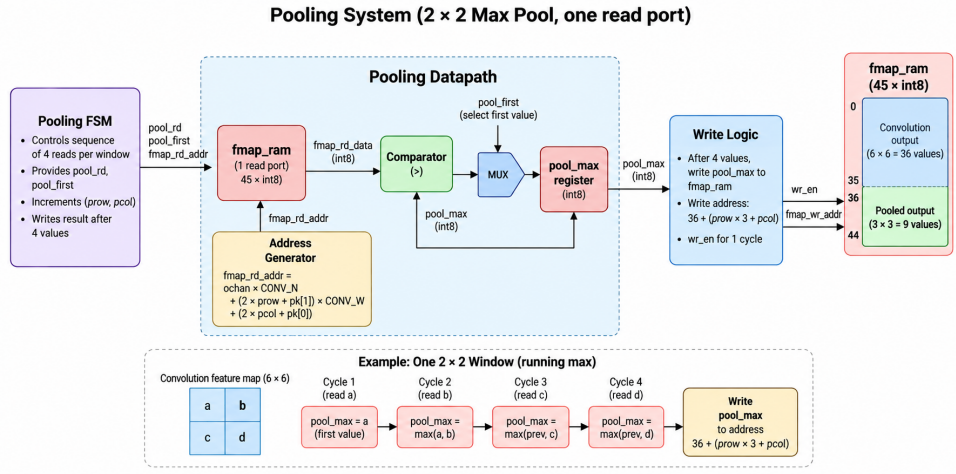


Figure 5: Architecture of the 2×2 max-pooling engine.

3.4 Control FSM

The Control FSM coordinates the complete inference process and controls memory accesses, MAC operations, pooling, and output generation. As shown in Figure 6, execution is divided into three major phases: convolution, pooling, and fully connected computation.

The FSM begins in the `S_IDLE` state and waits for a start command. During the convolution phase, image and weight data are read from memory, processed by the MAC unit, and written back as feature maps. Once all convolution outputs have been generated, the FSM transitions to the pooling phase, where 2×2 max-pooling is performed on the generated feature maps.

After all pooled outputs have been computed, the FSM enters the fully connected stage. The pooled values are processed using the same MAC datapath to generate the final classification outputs. Once all output classes have been evaluated, the FSM enters the `S_DONE` state and asserts the completion signal.

This centralized control approach enables efficient reuse of hardware resources while maintaining deterministic execution of the inference pipeline.

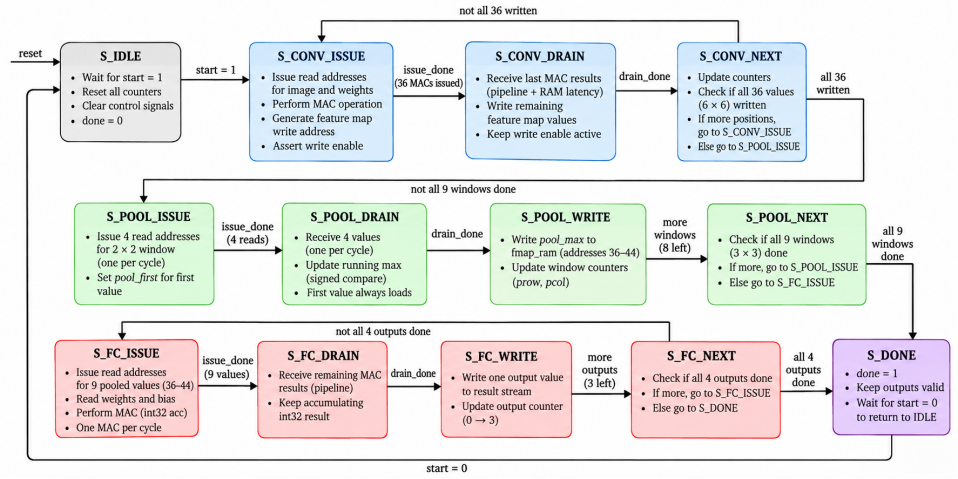


Figure 6: State transition diagram of the CNN accelerator control FSM.

4 Memory Map

The accelerator uses a memory-based architecture to store network parameters and intermediate results during inference. To minimize hardware complexity, weights, biases, feature maps, and pooled outputs are stored within a shared memory space using fixed address regions.

Figure 7 illustrates the logical organization of the memory. Separate address ranges are allocated for weights, biases, convolution feature maps, and pooled feature maps. This arrangement simplifies address generation and allows the FSM to access different data types using fixed offsets.

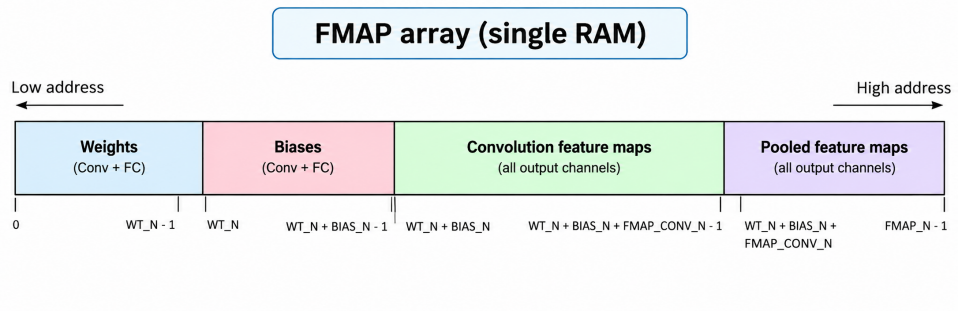


Figure 7: Logical memory organization used by the accelerator.

The memory regions are organized as follows:

- Convolution and fully connected weights

- Convolution and fully connected biases
- Convolution feature maps
- Pooled feature maps

This organization allows the accelerator to reuse a single memory structure while maintaining a simple addressing scheme throughout convolution, pooling, and fully connected computation.

4.1 Feature Map Memory

The feature map memory is implemented as a single read, single write RAM with registered read outputs. During convolution, generated feature maps are written into memory and later accessed by the pooling engine. The same memory is also used to store pooled outputs, reducing overall memory requirements.

The registered read interface introduces a one-cycle latency which is explicitly handled by the accelerator FSM through dedicated drain states.

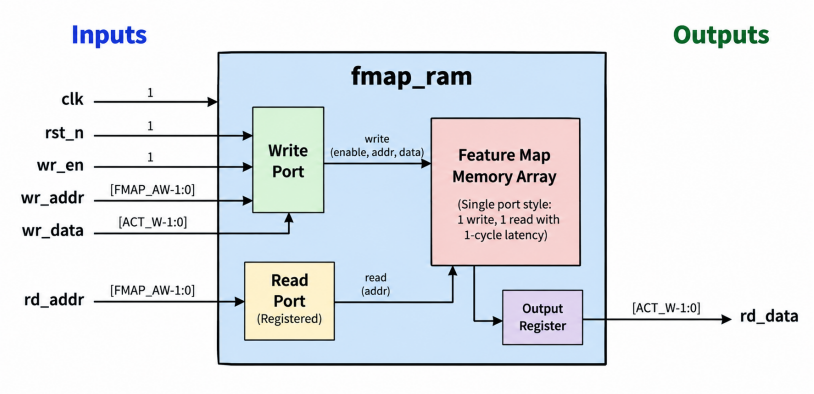


Figure 8: Feature map memory architecture.

4.2 Future CDC-Compatible Memory Architecture

The current implementation operates within a single clock domain and therefore does not require specialized clock domain crossing support. However, the original problem statement requires the AXI interface and accelerator core to operate at different clock frequencies.

A future version of the design can replace the current memory implementation with a dual-clock RAM structure, allowing independent read and write clock domains. In such an architecture, configuration data can be written from the AXI clock domain while the accelerator simultaneously reads data using its own clock.

This approach reduces CDC complexity for bulk data transfers and allows the accelerator subsystem to be integrated more easily into larger FPGA or SoC-based systems.

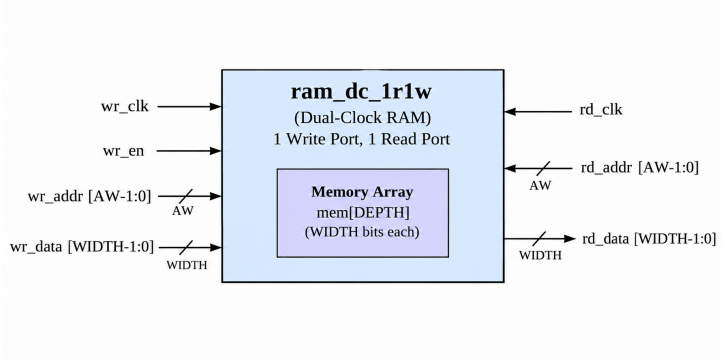


Figure 9: Dual-clock RAM architecture for future CDC integration.